Tree Learning: A Multi-Skill Contin Humanoid Robots

Yifei Yan[a], Linqi Ye[a,*]

[a]School of Future Technology, Shanghai University, No. 99 Shangda Road, Bao

ARTICLE INFO

Keywords:
Tree Learning
Reinforcement learning
Continual learning
Humanoid robot
Embodied intelligence

ABSTRACT

As reinforcement learnin efficiently expanding new embodied intelligence. E of-Experts (MoE) mode deployment difficult. To framework for humanoid tance mechanism, providi prevent catastrophic forg modulation and interpola level reward shaping strat experiments show that, in rewards across various re enabling seamless multi- performance and general a Super Mario-inspired in environment.

1. Introduction

The rapid development of embodied intelligence is driving autonomous robot technology from controlled laboratory environments toward general-purpose real-world scenarios [1, 2, 3]. As a typical representative of embodied intelligence, humanoid robots possess significant advantages in navigating unstructured environments—such as roads, staircases, and debris—due to their human-like gait and body structure [4, 5]. In recent years, reinforcement learning (RL)-based motion control methods [6, 7, 8, 9] have been evolving from single-task adaptation toward multi-skill incremental expansion, posing significant challenges in terms of generalization capability, continual learning efficiency, and system scalability.

In the domain of humanoid robots acquiring diverse skills, several recent advances have been achieved. BeyondMimic [10] realized flexible cross-task trajectory synthesis through a guided diffusion model. The KungfuBot series [11, 12] leveraged orthogonal mixture-of-experts (OMoE) models and bilevel optimization algorithms, enabling robots to imitate highly dynamic behaviors such as martial arts and dance. Any2Track [13] proposed a two-stage framework that substantially enhanced motion tracking robustness under multi-source disturbances.

However, three major technical gaps remain with respect to the core demand for continual incremental expansion. First, existing MoE-based approaches [14, 15, 16, 17] often require network topology adjustments when introducing new skills, while approaches represented by SONIC [18] tend

Project page: https://yyf-prog.github.io/Tree-learning/

*Corresponding author.
yyf20041019@shu.edu.cn (Y. Yan); yelinqi@shu.edu.cn (L. Ye)

inual Learning Framework for

Baoshan District, Shanghai, 200444, Shanghai, China

rning for humanoid robots evolves from single-task to multi-skill paradigms, new skills while avoiding catastrophic forgetting has become a key challenge in e. Existing approaches either rely on complex topology adjustments in Mixture- odels or require training extremely large-scale models, making lightweight To address this, we propose Tree Learning, a multi-skill continual learning noid robots. The framework adopts a root-branch hierarchical parameter inheri- viding motion priors for branch skills through parameter reuse to fundamentally forgetting. A multi-modal feedforward adaptation mechanism combining phase polation is designed to support both periodic and aperiodic motions. A task- strategy is also proposed to accelerate skill convergence. Unity-based simulation t, in contrast to simultaneous multi-task training, Tree Learning achieves higher s representative locomotion skills while maintaining a 100% skill retention rate, ulti-skill switching and real-time interactive control. We further validate the eralization capability of Tree Learning on two distinct Unity-simulated tasks: ed interactive scenario and autonomous navigation in a classical Chinese garden

to train extremely large-scale models, leading to computational costs that grow rapidly with the number of skills and making edge-side lightweight deployment difficult. Second, motion imitation-based methods [19, 20, 21] typically require full policy retraining when learning new motions, which readily leads to degradation of existing fundamental capabilities—the well-known catastrophic forgetting problem. Third, existing robust control strategies [22, 23, 24] are largely limited to parameter adaptation for basic gaits and cannot achieve cross-modal motion generalization at low training cost.

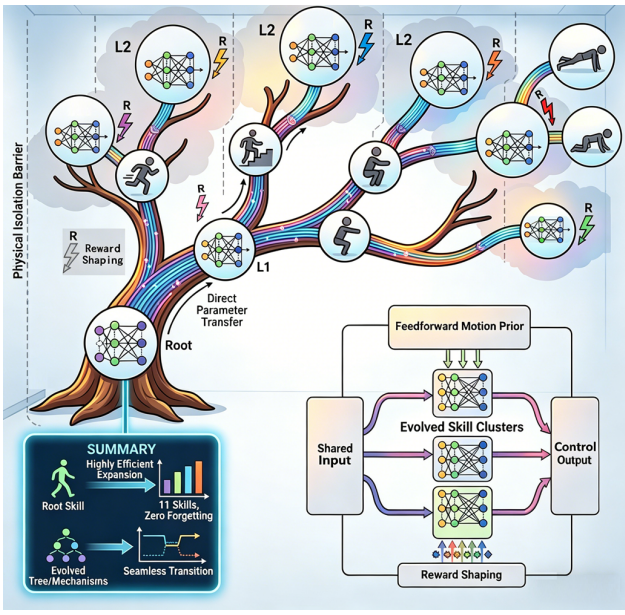


Figure 1: Tree Learning diagram.

Inspired by the mechanisms of prior inheritance and specialized evolution observed in biological evolution, this paper proposes Tree Learning (Figure 1)—a multi-skill continual learning framework based on hierarchical inheritance. The framework defines a pre-trained flat-ground basic gait as the root skill, while other skills inherit root skill parameters in a tree-structured manner layer by layer, referred to as branch skills. Through parameter inheritance and reward shaping, branch skills achieve rapid evolution. The main contributions of this paper are as follows:

(1) A tree-structured parameter inheritance architecture for continual learning is proposed. Through physically isolated sub-network clusters, catastrophic forgetting is fundamentally eliminated, enabling lossless incremental expansion of multiple skills.

(2) A low-cost skill iteration pipeline is designed, supporting incremental fine-tuning of specific branches and significantly reducing computational and time costs.

(3) Simulation validation is conducted on the Unitree G1 humanoid robot, achieving efficient expansion and seamless switching of 11 types of highly dynamic skills with real-time interactive control.

2. System Overview

As shown in Figure 2, the Tree Learning architecture defines flat-ground walking as the root skill and extends branch skills layer by layer based on a hierarchical inheritance mechanism. Continuing training from the root skill yields first-level branch skills: one-leg standing, stair climbing, lying prone, running, and squatting. From one-legged standing, a second-level branch skill—ball kicking—is derived through further training. From squatting, a second-level branch skill—two-footed jumping—is obtained. From the lying prone action, second-level branch skills including push-ups, crawling, and standing up are developed.

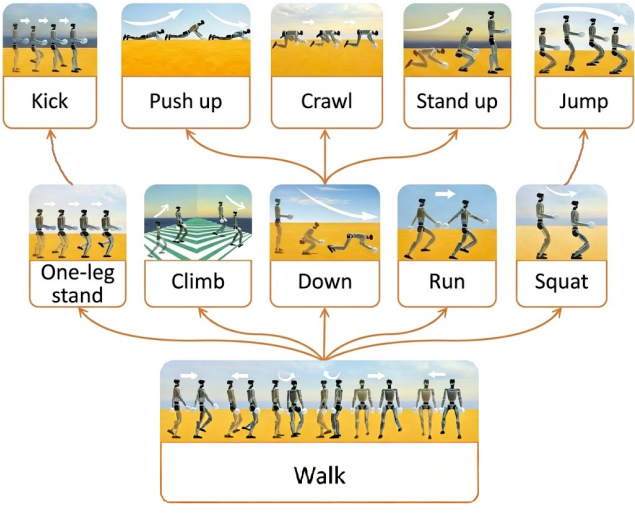


Figure 2: Tree Learning for Unitree G1.

The Tree Learning framework enforces consistency of the global state space and action representation by design.

Since the root skill and all branch sub-networks share identical sensor input interfaces and joint control output dimensions, and network switching is only performed between skills with overlapping action sequences and state spaces, the system maintains high consistency in motion continuity during dynamic model loading and skill switching. This consistency fundamentally avoids action discontinuities at the moment of neural network switching, thereby effectively preventing abrupt posture changes of the robot and enabling seamless, smooth transitions between heterogeneous locomotion modes.

In complex terrain or task scenarios requiring fine-grained control, the Tree Learning framework supports user intervention in skill switching and action adjustment through a lightweight input interface. For example, users can select target skill models via numeric keys 0/1/2/3 (e.g., 0 for flat-ground walking, 1 for running) and dynamically adjust action commands via W/S/A/D/Q/E letter keys (e.g., W/S for forward/backward, A/D for left/right turning, Q/E for lateral movement). Upon receiving user commands, the system combines state assessment to load the corresponding model and output action commands, forming a closed-loop interaction of user input–model execution–state feedback. This human-robot collaborative mode significantly enhances system adaptability in dynamic scenarios.

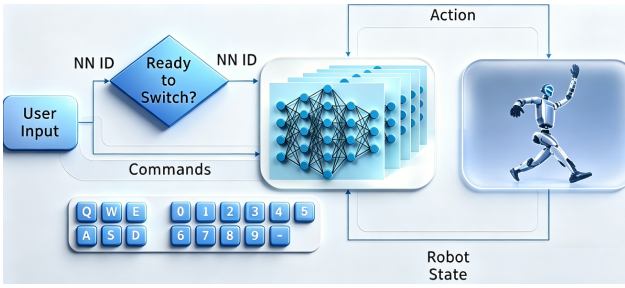


Figure 3: Interactive multi-skill control.

3. Methodology

3.1. Feedforward Action Design

For each skill, motion prior is used as a feedforward action to achieve a specific motion style. Different actions are realized through simple feedforward signals. For periodic and aperiodic actions, phase-based and interpolation-based methods are employed, respectively.

3.1.1. Phase modulation method

For periodic locomotion skills such as walking, running, stair climbing, and jumping, a cosine phase modulation method is used to design the feedforward action trajectory. The core mechanism employs gait phase activation factors u_1 and u_2 to schedule alternating motion of the left and right legs of the humanoid robot, providing a baseline action reference for reinforcement learning and reducing exploration cost.

The factors u_1 and u_2 represent the gait phase activation factors for the robot's left and right legs, respectively. Their computation relies on the gait cycle counter tp and the gait cycle period T_1.

The core formulas are given as follows:

$$\begin{cases} tp_0 = \begin{cases} tp, & 0 < tp \le T_1 \\ tp - T_1, & T_1 < tp \le 2T_1 \end{cases} \\ u_1 = \begin{cases} \frac{1}{2}\left(1 - \cos\left(\frac{2\pi tp_0}{T_1}\right)\right), & 0 < tp \le T_1 \\ 0, & T_1 < tp \le 2T_1 \end{cases} \\ u_2 = \begin{cases} 0, & 0 < tp \le T_1 \\ \frac{1}{2}\left(1 - \cos\left(\frac{2\pi tp_0}{T_1}\right)\right), & T_1 < tp \le 2T_1 \end{cases} \end{cases} \quad (1)$$

In Eq. (1), the gait time step counter tp auto-increments at each fixed time step (0.01 s) and resets to 0 upon reaching $2T_1$, enabling cyclic gait periodicity. A sub-counter tp_0 is used for single-leg phase cycle calculation. The cosine function maps the $[0, 2\pi]$ cosine period to $[0, 2]$, which is then divided by 2 to yield a $[0, 1]$ phase activation factor, ensuring smooth action scheduling.

The factors u_1 and u_2 implement left-right leg support phase separation through an alternating activation mechanism. By combining u_1 and u_2 with joint angle coefficients (where d_h denotes a dynamic adjustment coefficient and d_0 denotes a base angle offset), feedforward angle commands for the core lower-limb joints (hip, knee, and ankle) are generated.

$$u_{\text{total}}[i] = (d_h \cdot u_f + d_0) \cdot \text{sign}(\text{idx}[i]) \quad (2)$$

In Eq. (2), where u_f represents u_1 or u_2, idx is the index mapping array of lower limb joints, and sign(idx[i]) is used to control the joint movement direction. Finally, the periodic gait feedforward control of the robot is realized. Different branch skills adapt their characteristics by adjusting T_1, d_h, d_0, and the activation logic of u_1 and u_2.

3.1.2. Interpolation method

For aperiodic actions (e.g., squatting), a linear interpolation method is employed to generate reference joint angles. This method supports continuously adjustable action depth (e.g., linear mapping of squat amplitude between 0–100%). The feedforward commands provide the RL network with baseline trajectory boundaries that carry strong physical meaning, while the policy network is responsible for optimizing action along the feedforward signal, thereby substantially improving the learning efficiency of diverse motion skills.

Table 1
Basic reward components

No.	Reward Term	Mathematical Expression
1	Velocity tracking	$r_{\text{vel}} = 1 - 2\lvert v_x - v_x^{\text{cmd}}\rvert - 2\lvert v_z - v_z^{\text{cmd}}\rvert$
2	Angular velocity tracking	$r_{\text{wel}} = 1 - 2\lvert w_y - w_y^{\text{cmd}}\rvert$
3	Center of mass height	$r_{\text{height}} = 2(h - h_{\text{target}})$
4	Orientation constraint	$r_{\text{ori}} = -0.1\lvert\theta_{\text{roll}}\rvert - 0.1\lvert\theta_{\text{pitch}}\rvert$
5	Survival reward	$r_{\text{live}} = 1$
6	Action energy penalty	$r_{\text{penalty}} = -0.2 \sum_{i \in J} \lvert u_i\rvert$

3.2. Parameter Inheritance

Parameter inheritance is primarily embodied in the training phase. The root skill (flat-ground walking) is first trained to completion, equipping the robot with stable basic locomotion capability. All branch skills are initialized from their parent skill network parameters, thus avoiding the efficiency loss associated with training from scratch.

Branch networks fully inherit the weights of the parent network as their initialization parameter value. This not only preserves the state-perception features and physical dynamics representations extracted in the hidden layers, but also avoids the random exploration inherent in training from scratch. During inference, to satisfy the memory constraints of edge devices and to fundamentally eliminate gradient interference from multi-task joint optimization, the framework employs physically isolated sub-network clusters. By sharing the same state space and similar distribution manifolds, the system ensures continuity of action commands during neural network model switching (models are in ONNX format), thereby achieving incremental expansion of multiple skills.

3.3. Reward Shaping

To accelerate skill convergence and improve execution stability, a set of basic reward components is first designed, as shown in Table 1.

In Table 1, v_x is the robot's lateral velocity; v_x^{cmd} is the robot's target lateral velocity; v_z is the robot's forward velocity; v_z^{cmd} is the robot's target forward velocity; w_y is the robot's torso yaw angular velocity about the Y-axis; w_y^{cmd} is the preset target yaw angular velocity about the Y-axis of the robot's torso; h_{target} is the robot's torso target height; θ_{roll} is the body roll angle; θ_{pitch} is the body pitch angle; u_i represents the output amplitude of the control command for the corresponding joint. To control redundant movements of non-essential joints, the set of penalty joints is defined as $J = \{1, 2, 5, 7, 8, 11\}$.

Since different skills have distinct requirements and preferences, the basic reward components can be recombined or supplemented with preference-specific reward terms to accommodate different skills.

Table 2
Core hyperparameter configuration of PPO

No.	Parameter Name	Value
1	Time steps per update	1000
2	Batch size	2048
3	Replay buffer size	20480
4	Number of epochs	3
5	Reward scale	1.0
6	Discount factor γ	0.995
7	GAE coefficient λ	0.95
8	Learning rate	3×10^{-4}
9	Clipping parameter ϵ	0.2
10	Entropy coefficient β	5×10^{-3}

3.4. Training Configuration

3.4.1. Network Architecture and RL Algorithm

This study is based on the Unity ML-Agents framework, employing the Proximal Policy Optimization (PPO [25]) algorithm for reinforcement learning training. For the network architecture, the actor policy network adopts a 3-layer fully connected multilayer perceptron (MLP) with 512 hidden units per layer and normalization of input state features. The policy output is set as a Gaussian distribution to ensure non-deterministic exploration. The critic value network uses a 2-layer fully connected structure (hidden layer dimension of 128) without state normalization. During training, model weights are saved every 4×10^5 steps to monitor convergence trends.

3.4.2. Reinforcement Learning Hyperparameters

The total training steps for different skills are adaptively adjusted according to their convergence difficulty. The core hyperparameters of the PPO algorithm employ a linear decay strategy to balance exploration in the early stages and exploitation in the later stages. The detailed configuration is presented in Table 2.

3.4.3. Progressive Curriculum Learning

For complex skills with strong terrain constraints, such as stair climbing, direct training in the target staircase environment is prone to becoming trapped in local optima. To address this, an environment-variable-driven curriculum learning mechanism is introduced. Specifically, staircase step height is designated as the curriculum difficulty metric and divided into four progressive stages (Lesson 1 through Lesson 4, with heights set to 0.01 m, 0.05 m, 0.10 m, and 0.15 m, respectively). During training, when the robot's smoothed cumulative reward at the current stage reaches a preset threshold, the environment automatically advances to the next difficulty level. This mechanism effectively alleviates the sparse reward problem on unstructured terrain, enabling smooth transition and efficient convergence of the robot's obstacle-crossing capabilities.

4. Comparison with Multi-Task Baseline

This section selects 6 representative skills from the 11 skill types for quantitative comparison between Tree Learning and simultaneous multi-task training (Multi-task). In the experiment, the multi-task baseline uses a single policy network to train all 6 skills simultaneously, whereas Tree Learning trains each skill independently. To ensure comparable computational budgets, both experimental groups are set to a total of 10^7 environment steps.

The experimental results are presented in Figures 4–9. As observed, Tree Learning achieves higher final rewards than the multi-task baseline across all 6 selected skills. Specifically, for the highly dynamic skills Jump and Run, Tree Learning attains stable rewards of 990 and 6138, respectively, while the multi-task baseline achieves only 99 and 609. Figure 10 presents a radar chart of stable converged reward distributions across the 6 skills, clearly showing that the final reward of the Run skill under Tree Learning is substantially higher than that of the multi-task baseline.

These results indicate that simultaneous multi-task training suffers from significant gradient conflicts: the substantial differences among reward functions for different skills lead to negative transfer during optimization, rendering some highly dynamic skills (e.g., Jump) virtually unable to converge effectively. In contrast, Tree Learning fundamentally avoids gradient interference through physically isolated independent sub-networks, allowing each skill to fully leverage the neural network weights from the root skill to achieve efficient and stable convergence.

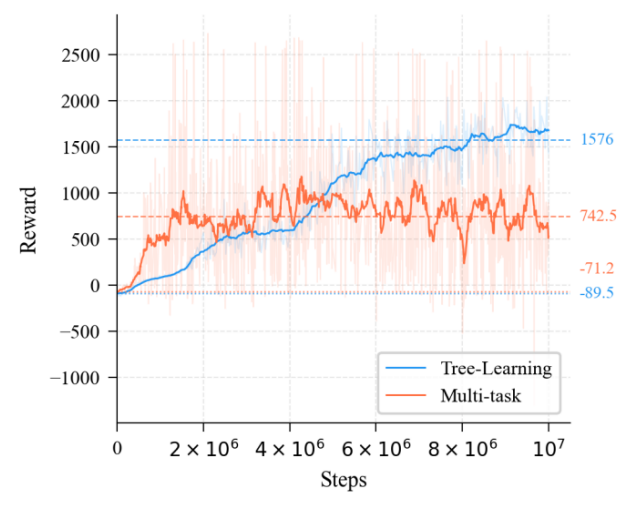


Figure 4: Reward comparison of walk skill.

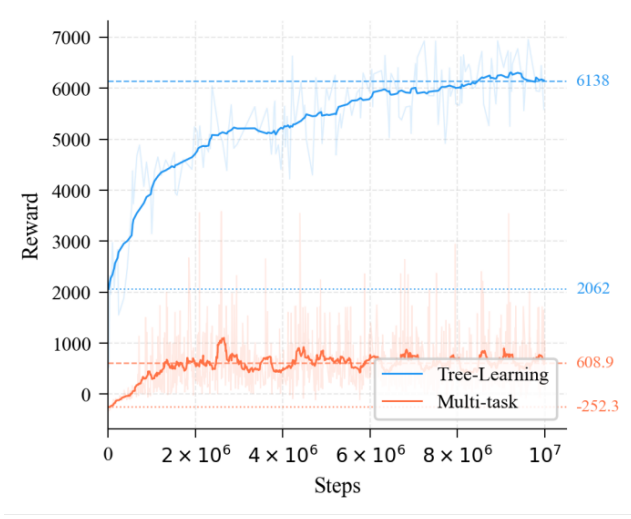


Figure 5: Reward comparison of run skill.

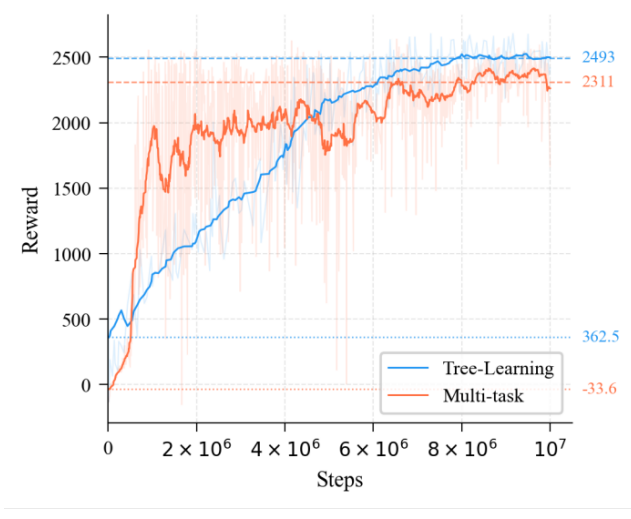


Figure 6: Reward comparison of squat skill.

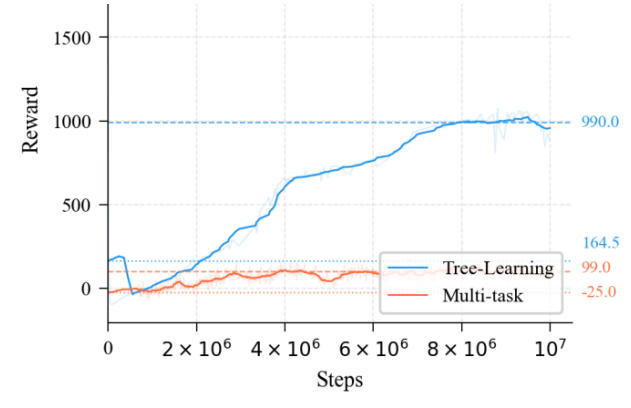


Figure 7: Reward comparison of jump skill.

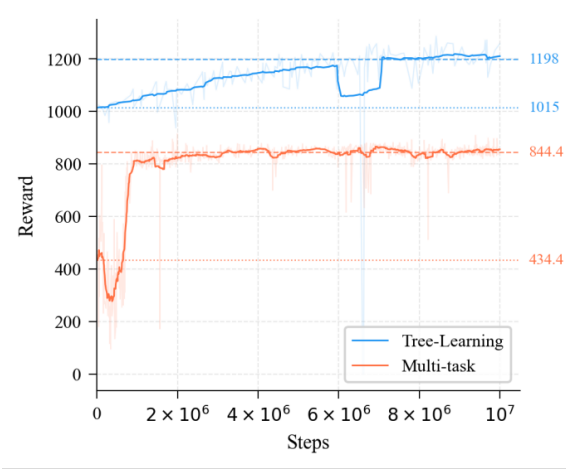


Figure 8: Reward comparison of crawl skill.

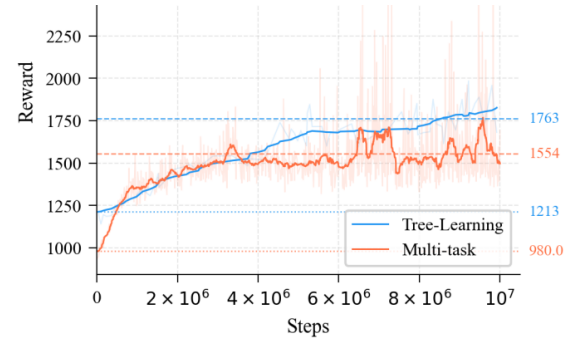


Figure 9: Reward comparison of one-leg stand skill.

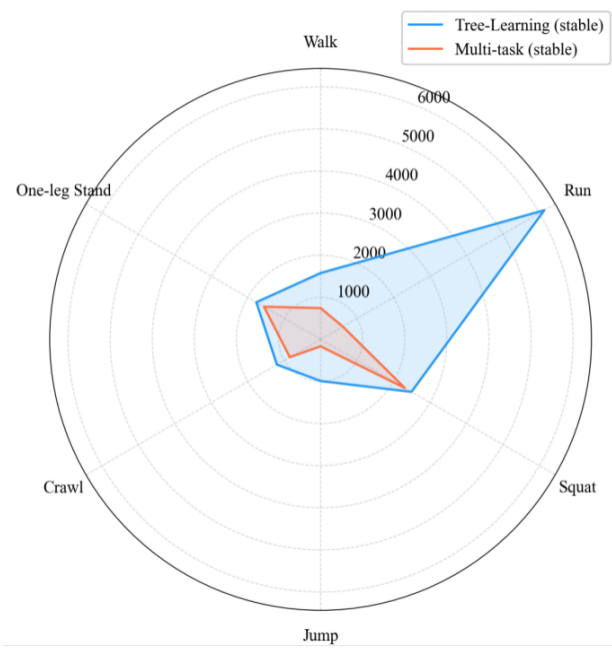


Figure 10: Final reward comparison.

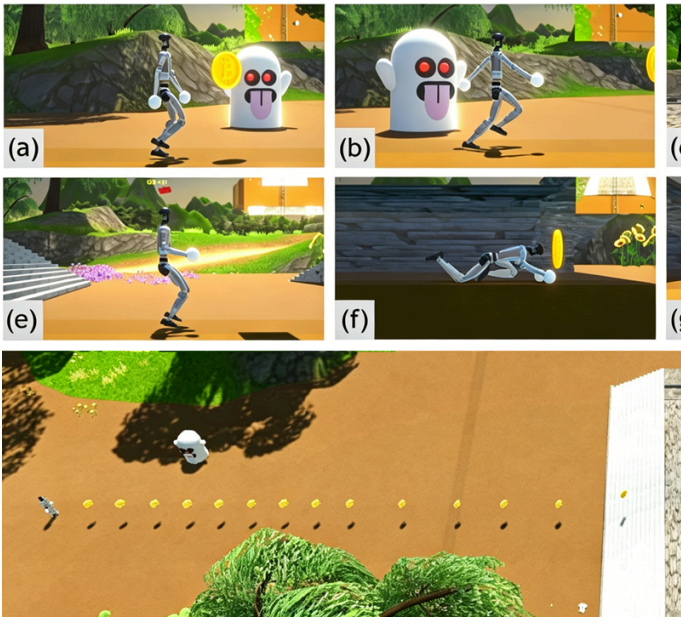


Figure 11: Super Mario simulation scene. The robot sequentiall climbing up and down stairs, (d) lying prone, (e) crawling throu collect coins or gifts along the way. Finally, (h) the robot kicks a

5. Super Mario Scenario Experiment

5.1. Simulation Environment and Task Setup

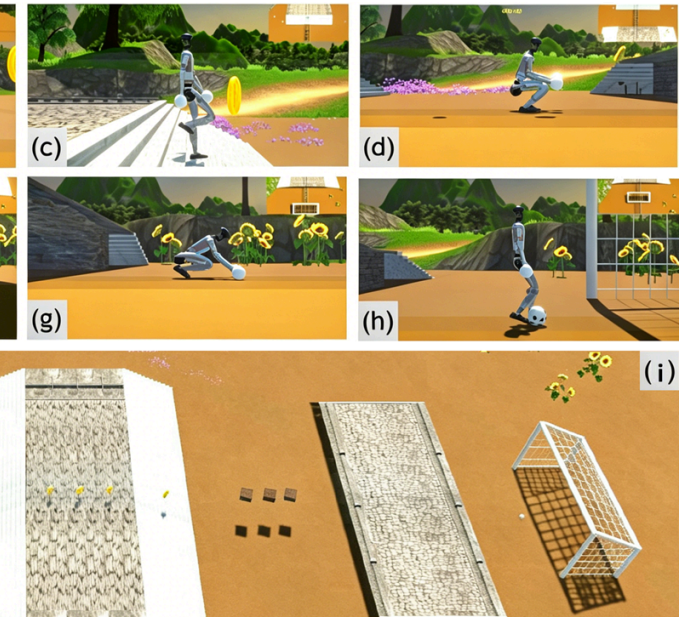
The simulation environment was built using Unity 2022. 3.62f3 LTS. The Unitree G1 humanoid robot (23-DoF version) was selected as the experimental platform, configured with 12 lower-limb joints, 1 waist joint, and 10 upper-limb joints. Multi-skill expansion was achieved through coordinated control of lower-limb joint motion and optimization of upper-limb joint motion. To increase the interestingness of the experiment and verify the comprehensive performance of multi-skill capabilities in continuous complex tasks, the experimental scenario was designed in the style of a Super Mario game, as shown in Figure 11. To prevent the robot from deviating laterally, two transparent walls were added to constrain movement to the forward-backward direction.

5.2. Manual Switching Control

The Super Mario test scene contains multiple interactive constraints including pursuers, coins, and complex terrain, posing high demands on the temporal sequencing of the humanoid robot's actions. Experimental observations revealed that the robot must complete multiple action transitions within extremely narrow action windows. Through manual switching control via keyboard, a typical full-process task sequence was successfully executed, validating the physical feasibility of the Tree Learning framework in handling highly dynamic and tightly coupled tasks. See Supplementary Video 1 for details.

5.3. Automatic Switching Control

For convenience, we also developed a method to enable the robot to automatically switch from walking to running



tially performed (a) walking, (b) running to escape a ghost, (c) rough a tunnel, (f) standing up, and (g) jumping to hit boxes to a ball into the goal and wins. (i) is the global view.

and subsequent complex action sequences based on environmental cues. To quantitatively assess switching continuity, center-of-mass (CoM) height and forward velocity curves were extracted during the automatic switching process.

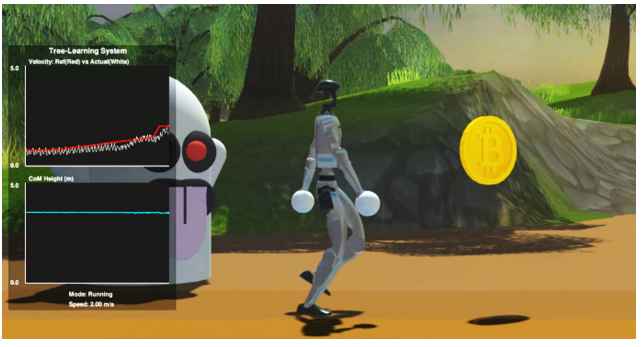


Figure 12: Ghost chasing scene.

As shown in Figure 12, when the robot detects a ghost pursuit and triggers an acceleration command, the CoM velocity curve exhibits notable continuity, smoothly transitioning from 0.8 m/s to 2.0 m/s without any step-like jumps or acceleration spikes throughout the switching interval. This smooth locomotion performance physically validates the effectiveness of the state space consistency design proposed in Section 2—namely, that physical quantities from different skill models can naturally align at the moment of switching. The entire automatic task flow demonstrates that the framework possesses high kinematic coherence when handling heterogeneous skill sequences. See Supplementary Video 2 for more details.

6. Autonomous Navigation Experiment

To further validate the robustness and control quality of the Tree Learning framework under a prolonged continuous task, an autonomous navigation task was designed in a classical Chinese garden scenario. Unlike the manual and automatic switching experiments in previous section, which focused on verifying switching feasibility, this experiment concentrates on a comprehensive evaluation of the robot during nearly 240 seconds of continuous locomotion, encompassing path planning, velocity control, gait switching, posture balancing, and forward-turn exchange. The results are given in Supplementary Video 3. The sampling rate was set to 50 Hz, resulting in about 12,000 recorded frames.

The navigation test scenario includes buildings, streets, and bridges, as shown in Figure 13. Based on the skills learned by Tree Learning, we adopted a navigation workflow as shown in Figure 14. Figure 13 shows the top-view trajectory of the robot. Starting from the starting point (green dot), the robot follows the path from the planner to sequentially approach multiple operator-specified target points and finally reach the end point (red square). The trajectory covers an area of approximately 30 m $\times$ 30 m. Multiple turns and obstacle avoidance behaviors are observed along the path, indicating that the navigation system can effectively plan feasible paths in complex scenarios with various terrains.

During the experiment, the operator clicks random target points in the scenario in real time. After the robot reaches the current target, the next target point is clicked immediately. During the task, the robot sequentially experiences three motion modes within 240 seconds: walking (Walk), running (Run), and stair climbing (Stair). The gait mode switching is automatically performed by the navigation system according to environmental conditions: when the target distance is far, the system automatically switches to Run mode to improve locomotion efficiency; when stairs are detected ahead, the system automatically switches to Stair mode; in other cases, Walk mode is used by default.

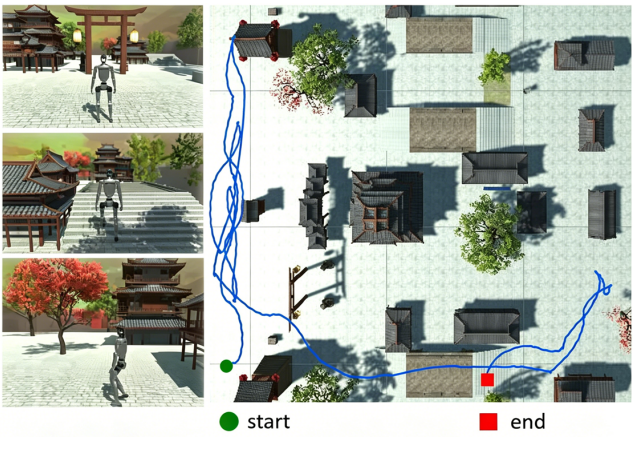


Figure 13: Autonomous navigation task. The left shows snapshots during the task. The right shows the top-down trajectory of the robot.

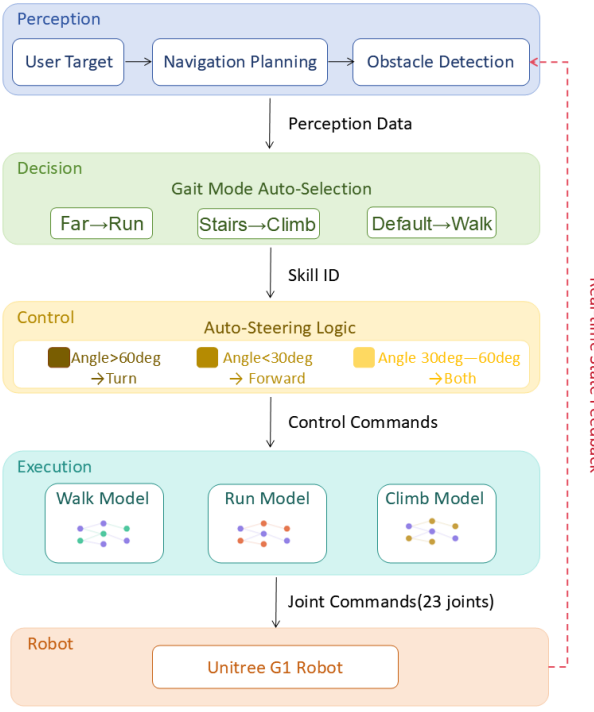


Figure 14: Autonomous navigation workflow.

6.1. Navigation Performance Analysis

Figure 15 presents the position curves of the robot's center of mass on the three coordinate axes over time. The X-axis position continuously rises from 0 to approximately 27 m after 120 s, corresponding to the robot's long-distance navigation phase toward a far target. The Z-axis position exhibits reciprocating fluctuations, reflecting the robot's path between multiple target points. Of particular interest is the Y-axis (height) data: during the flat-ground phase from 0 to 160 s, the center-of-mass height remains stable at approximately 0.78 m with minimal fluctuations. In the interval from 170 to 190 s, the center-of-mass height rises sharply from 0.78 m to approximately 1.78 m and then stabilizes at the top of the stairs, proving that the robot successfully climbed a stair of about 1 m in height. The time interval of this height jump matches exactly with the Stair mode activation interval in Figure 16, verifying the physical effectiveness of gait switching from the spatial position dimension. At the end of the experiment (around 235 s), the Y-axis shows a slight rise again, corresponding to the second short stair climbing.

Figure 16 quantitatively presents the navigation performance metric over time. The upper figure shows the target distance curve, which exhibits a typical "sawtooth" descending pattern: whenever the distance approaches zero, the system determines that the current target has been reached, the operator specifies a new target immediately, and the distance jumps accordingly. The green vertical lines mark the events of reaching each target. During the entire experiment, the robot successfully reached 22 target points with only one stuck recovery, demonstrating extremely high navigation reliability. Notably, a long-distance navigation segment (with

a distance peak close to 20 m) appears around 120 s, which is a typical scenario where the system automatically switches to Run mode based on the target distance. The lower figure shows the obstacle factor curve, which reflects the degree to which obstacles in front suppress the robot's speed: a value of 1.0 indicates no obstacles ahead, while values close to 0 indicate detection of close-range obstacles. In the early half of the experiment (0–120 s), the obstacle factor remains at 1.0, indicating that this area is dominated by flat ground or open corridors. In the intervals of 140–170 s and 200–240 s, the obstacle factor drops multiple times, corresponding to the robot entering stairwells or narrow passages and performing obstacle avoidance deceleration.

Figure 17 presents the statistic of the navigation experiment in three forms: pie chart, bar chart, and histogram. The gait distribution pie chart shows that Walk mode accounts for 87.8%, Stair mode for 10.1%, and Run mode for 2.0%, which is consistent with the expected design of mainly normal walking, automatic acceleration for long-distance targets, and stair climbing. The navigation bar chart visually compares the completion of 22 target points with only one stuck recovery, verifying the high reliability of the system in multi-terrain environments. The forward velocity histogram shows a bimodal distribution, with the main peak near 0.0 m/s (corresponding to standing or deceleration phases) and the secondary peak in the range of 0.8–1.0 m/s (corresponding to steady-state forward movement). This distribution is highly consistent with the designed auto-steering control logic.

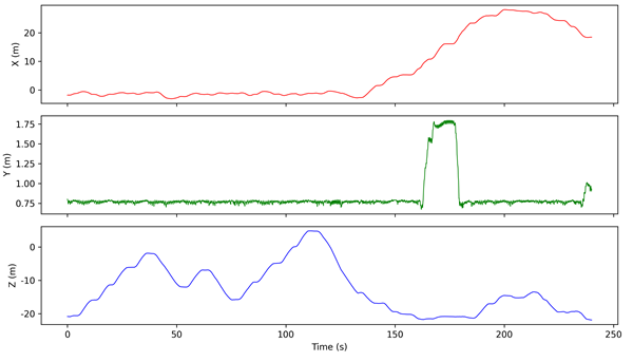


Figure 15: Robot CoM position curve.

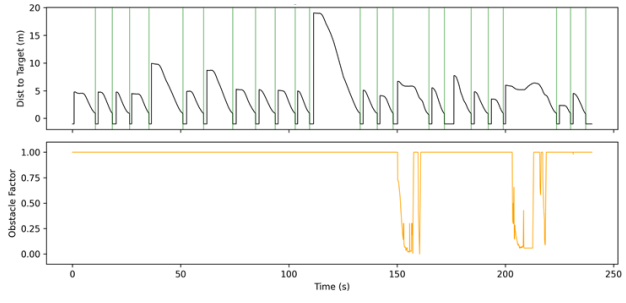


Figure 16: Autonomous navigation metric.

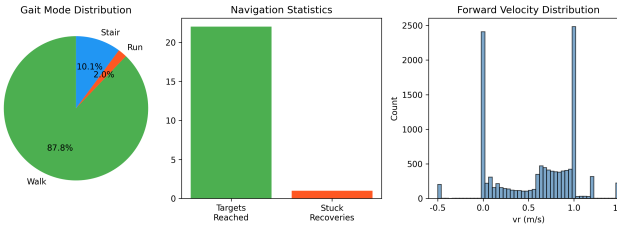


Figure 17: Autonomous navigation statistic.

6.2. Motion Control Analysis

Figure 18 shows the time series of control commands v_r (forward velocity), w_r (turning angular velocity). The v_r curve exhibits a distinct "step-pulse" alternating pattern: v_r stabilizes at 0.8–1.0 m/s during straight segments and drops rapidly to near zero during turning segments. Correspondingly, the w_r curve fluctuates significantly during turning segments (peak value ± 1.0 rad/s) and remains small during straight segments. Notably, v_r shows a peak of 1.5 m/s at around 120 s, corresponding to the high-speed forward phase when Run mode is activated. The inverse coupling relationship between v_r and w_r visually verifies the primary auto-steering control logic of forward and turning in the navigation system: the forward velocity is preferentially reduced during turning to ensure steering accuracy, while the forward efficiency is maximized during straight-line movement.

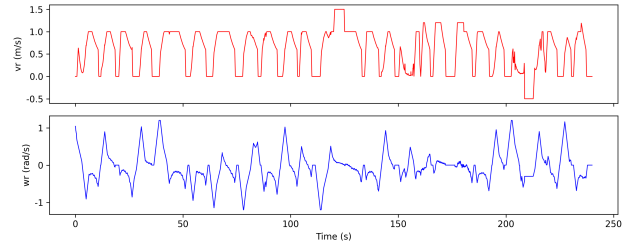


Figure 18: Control commands(vr/wr).

To further reveal the inherent structure of the forward-turn mutual exclusion relationship, Figure 19 plots the v_r-w_r phase diagram with target angle as the color mapping. The gray dashed line in the figure represents the speed-turn interlock boundary, which is determined by the parameters in the controller. Two distinct regions can be clearly observed in the figure: when the target angle is large (warm color, angle $> 60°$), the data points are concentrated in the region of low v_r and high $|w_r|$ (marked as “Turn-dominant” in the figure), indicating that the robot prioritizes turning to align with the target direction; when the target angle is small (cool color, angle $< 30°$), the data points are concentrated in the high v_r and low $|w_r|$ region (marked as “Forward-dominant” in the figure), indicating that the robot has basically aligned with the target and accelerates forward. The entire data distribution is enclosed by the interlock boundary, and the control commands are always within the physical safety envelope regardless of the gait mode.

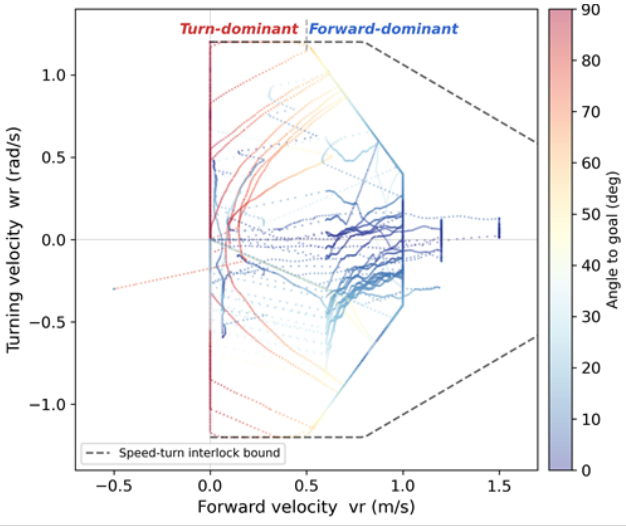


Figure 19: vr-wr phase portrait.

6.3. Gait Switching and Posture Stability

Figure 20 shows the temporal switching of gait modes during the 240-second experiment in the form of a color band diagram. The robot remained in Walk mode throughout the 0–120 s period, conducting stable locomotion in the flat environment. Then, at approximately 120 s, as the operator specified a farther target point, the system automatically switched to Run mode (lasting about 5 s) for acceleration, and then returned to Walk mode. In the 160–180 s interval, the robot reached the stair area, and the system automatically switched to Stair mode to adapt to the step terrain, returning to Walk mode after passing the stairs. At the end of the experiment (around 235 s), the robot encountered stairs again and briefly entered Stair mode. The gait switching sequence throughout the experiment was highly consistent with terrain changes and target distances, and all switching was completed automatically by the navigation system, indicating that the multi-skill switching logic under the Tree Learning framework has good environmental adaptability in autonomous navigation scenarios.

Figure 21 presents the posture stability data of the robot's torso. The upper figure shows the time curves of roll and pitch angles. During the flat-ground walking phase, both roll and pitch angles were controlled within about $\pm 2.5°$, demonstrating high posture stability. In the stair interval (160–180 s), the Roll angle fluctuations increased to approximately $\pm(5–7)°$, caused by periodic disturbances from the stair steps, but remained within a safe range without falls or severe instability. This further verifies the design concept of a unified state space shared by all skill models in the Tree Learning framework: different branch networks can achieve continuous transition of action commands at the moment of switching, ensuring posture robustness throughout autonomous navigation.

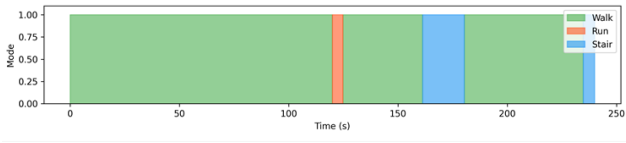


Figure 20: Gait mode distribution along path.

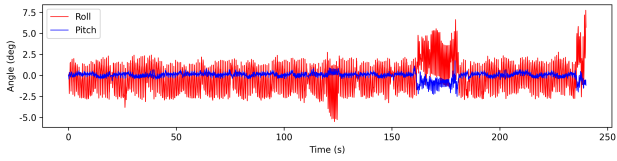


Figure 21: Torso orientation curve.

7. Discussion

7.1. Mechanism Effectiveness Analysis

The Tree Learning framework achieves both enhanced multi-skill expansion efficiency and high motion stability, primarily owing to the synergistic effect of its underlying design components. First, the parameter inheritance mechanism reuses the neural network weights embedded in the root skill (e.g., flat-ground walking). This approach of learning by building upon established foundations substantially reduces the blind exploration range in the high-dimensional state-action space, significantly lowering the exploration cost and sample complexity for new skills. Second, the feedforward action and reward shaping strategy provides targeted dense guidance for different branch skills, not only accelerating network convergence but also ensuring action reasonableness under physical constraints. Third, the multi-modal action adaptation mechanism combining phase modulation and trajectory interpolation enables the framework to accommodate both periodic gaits and aperiodic large-range motions, substantially extending the system's applicability boundary.

7.2. Summary of Core Advantages

Compared with conventional reinforcement learning-based motion control approaches, the Tree Learning framework proposed in this paper demonstrates advantages as follows:

(1) High training efficiency: Owing to hierarchical parameter reuse, the converged rewards of Tree Learning consistently surpass those of the multi-task baseline when expanding new skills.

(2) Avoidance of catastrophic forgetting: Through the physically isolated tree-structured network storage topology, the system achieves decoupling of old and new skills, with a retention rate of 100% for existing fundamental skills, resolving the negative transfer problem in single-network multi-task learning.

(3) Broad scenario adaptability: The framework successfully covers a wide skill spectrum ranging from simple flat-ground locomotion to complex environments (e.g., stairs, tunnel) and highly dynamic interactive motion (e.g., ball kicking, jumping).

(4) Practical interactivity: Through a lightweight global clock and state relay logic, the system supports smooth, ultra-low-latency dynamic skill switching under user commands, with no posture transients or fall instabilities observed.

7.3. Limitations and Future Directions

Despite the promising results achieved in simulation, this study has certain limitations that point to directions for future work. On one hand, the current framework has been validated only in the Unity simulation environment, without fully accounting for the complexities of real motor dynamics, sensor noise, and contact friction—constituting an objective Sim-to-Real gap. In future work, we plan to add domain randomization and implicit state estimation techniques to pursue zero-shot transfer deployment to physical humanoid robots. On the other hand, long-period complex motions (e.g., dance) have not yet been addressed. The next step will involve developing automated motion retargeting and keyframe auto-extraction algorithms based on human video demonstrations, to further improve the construction efficiency of complex skills.

8. Conclusion

This paper addresses the prevalent pain points of low training efficiency and susceptibility to catastrophic forgetting in multi-skill learning for humanoid robots by proposing Tree Learning, a multi-skill continual learning framework. The framework innovatively adopts a root-branch hierarchical network architecture, achieving efficient expansion of 11 diverse motor skills for humanoid robots through prior parameter inheritance, task-level reward shaping, and multi-modal action feedforward adaptation. Simulation experiments demonstrate that, compared with the simultaneous multi-task training baseline, Tree Learning not only improves the converged rewards for new skills but also achieves seamless human-robot interactive switching of cross-terrain, cross-modal action commands. This work fundamentally avoids the gradient conflicts arising from complex skill accumulation from a mechanistic perspective, providing an efficient system-level solution for continual learning of embodied intelligent agents.

Acknowledgements

This work was supported by the Science and Technology Commission of Shanghai Municipality (24511103304).

CRediT authorship contribution statement

Yifei Yan: Methodology, Software, Writing - original draft, Visualization. **Linqi Ye:** Conceptualization, Supervision, Project administration, Funding acquisition, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Wu, J., Xin, G., Qi, C., Xue, Y., 2023. Learning robust and agile legged locomotion using adversarial motion priors. IEEE Robotics and Automation Letters 8, 4975–4982. doi:10.1109/LRA.2023.3290509.

[2] Zhu, S., Huang, R., Mou, L., et al., 2024. Robust robot walker: Learning agile locomotion over tiny traps. arXiv preprint arXiv:2409.07409 .

[3] Lee, J., Hwangbo, J., Wellhausen, L., et al., 2020. Learning quadrupedal locomotion over challenging terrain. Science Robotics 5, eabc5986.

[4] Ha, S., Lee, J., van de Panne, M., et al., 2025. Learning-based legged locomotion: State of the art and future perspectives. The International Journal of Robotics Research 44, 1396–1427.

[5] Liu, L.M., Tian, Y.T., Li, J.F., et al., 2011. Switch control of passive walking robot under variable road conditions. Control and Decision 26, 1203–1208.

[6] Xie, B., Chen, Y.L., Liu, X.L., et al., 2025. Overview of the intelligent robot training platform. Space Electronic Technology 22, 1–9.

[7] Gou, G.Z., Guo, M., 2025. Review of domestic humanoid robot development in 2024. Robot Technique and Application 9, 5–13. doi:10.3969/j.issn.1004-6437.2025.02.007.

[8] Liu, W.L., Li, B., Hou, L.D., et al., 2022. Review of quadruped robot research based on deep reinforcement learning. Journal of Qilu University of Technology 32, 67–74.

[9] Zhang, W.Y., Xia, D.W., Chang, G.Y., et al., 2025. A dual vision-guided mobile robot control approach for multi-target path planning and intelligent pickup. Robotics and Autonomous Systems 190, 104993.

[10] Liao, Q., Truong, T.E., Huang, X., et al., 2025. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. arXiv preprint arXiv:2508.08241 .

[11] Xie, W., Han, J., Zheng, J., et al., 2025. Kungfubot: Physics-based humanoid whole-body control for learning highly-dynamic skills. arXiv preprint arXiv:2506.12851 .

[12] Han, J., Xie, W., Zheng, J., et al., 2025. Kungfubot2: Learning versatile motion skills for humanoid whole-body control. arXiv preprint arXiv:2509.16638 .

[13] Zhang, Z., Guo, J., Chen, C., et al., 2025. Track any motions under any disturbances. arXiv preprint arXiv:2509.13833 .

[14] Obando-Ceron, J., Sokar, G., Willi, T., et al., 2024. Mixtures of experts unlock parameter scaling for deep rl. arXiv preprint arXiv:2402.08609 .

[15] Yang, C., Yuan, K., Zhu, Q., et al., 2020. Multi-expert learning of adaptive legged locomotion. Science Robotics 5, eabb2174.

[16] Huang, R., Zhu, S., Du, Y., et al., 2025. Moe-loco: Mixture of experts for multitask locomotion, in: 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 14218–14225.

[17] Cheng, G., Dong, L., Cai, W., et al., 2023. Multi-task reinforcement learning with attention-based mixture of experts. IEEE Robotics and Automation Letters 8, 3812–3819.

[18] Luo, Z., Yuan, Y., Wang, T., et al., 2025. Sonic: Supersizing motion tracking for natural humanoid whole-body control. arXiv preprint arXiv:2511.07820 .

[19] Peng, X.B., Abbeel, P., Levine, S., et al., 2018. Deepmimic: Example-guided deep reinforcement learning of physics-based character skills. ACM Transactions on Graphics 37, 1–14.

[20] Li, Z., Peng, X.B., Abbeel, P., Levine, S., Berseth, G., Sreenath, K., 2024. Reinforcement learning for versatile, dynamic, and robust bipedal locomotion control. arXiv preprint arXiv:2401.16889 .

[21] Zhang, L.J., Tang, L., Liu, L., 2025. Whole-body motion strategy intelligent generation method for multi-skilled humanoid robots. Aerospace Control and Application 51, 28–40.

[22] Zhang, X., Xiao, Z., Zhang, Q., et al., 2024. Synloco: Synthesizing central pattern generator and reinforcement learning for quadruped locomotion, in: 2024 IEEE 63rd Conference on Decision and Control (CDC), pp. 2640–2645.

[23] Bellegarda, G., Shafiee, M., Ijspeert, A.J., 2024. Visual cpg-rl: Learning central pattern generators for visually-guided quadruped locomotion, in: 2024 IEEE International Conference on Robotics and Automation (ICRA), pp. 1420–1427.

[24] Chen, L., Cui, R., Yan, W., et al., 2024. Terrain-adaptive locomotion control for an underwater hexapod robot: Sensing leg-terrain interaction with proprioceptive sensors. IEEE Robotics & Automation Magazine 31, 41–52.

[25] Schulman, J., Wolski, F., Dhariwal, P., et al., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 .